

Malware Analysis using AI

Dr Mukti Padhya

Assistant Professor

SCSDF,NFSU

Malware Detection and Classification

- **Traditional AV solutions**
 - Signature based
 - Heuristic based
- **Requires Malware Analysis**
 - Static analysis (faster but evades detection if malware obfuscated or concealed)
 - Dynamic analysis
- **Need for ML approach**
 - A large amount of data due to large attacks leads to high computation for searching and matching used in traditional AV approach

Malware Detection and Classification

- Traditionally, AV solutions relied on signature-based and heuristic-based methods.
- A signature is an algorithm or hash that uniquely identifies a specific malware while heuristics are a set of rules determined by experts after analyzing the behavior of malware.
- However, both approaches require the malware to be analyzed prior to the definition of these rules and heuristics.
- The goal of malware analysis is to provide information about the characteristics, purpose and behavior of a given piece of software.
- There are two types of analysis: (1) static analysis and (2) dynamic analysis.
- Static analysis involves examining an executable without execution.
- Dynamic analysis involves examining the behavior of the executable by running it.
- Static analysis is faster but, if malware is successfully concealed using code obfuscation techniques, it could evade detection. Contrarily, code obfuscation techniques and polymorphic malware hardly evades dynamic analysis as it monitors and analyzes the runtime execution of a program.

Taxonomy of Malware

- **Adware**
- **Backdoor**
- **Bot**
- **Downloader**
- **Launcher**
- **Ransomware**
- **Rootkit**
- **Spyware**
- **Trojan**
- **Virus**
- **Worm**

Taxonomy of Malware

- Adware. Malware designed to automatically generate online advertisements. This type of malware generates revenue for its developer by displaying advertisements on the user interface or the screen.
- Backdoor. Computer software that is designed to bypass a system's security mechanism and install itself on a computer to allow the attacker to access it.
- Bot. Software created to automatically perform specific operations such as DDoS attacks or distribute other malware. Bots are part of a botnet, a network of interconnected devices, which are controlled using command and control (C&C) software.
- Downloader. A downloader program's purpose is to download and install additional malicious programs.
- Launcher. A launcher is a computer program designed to stealthily launch other malicious programs.

Taxonomy of Malware

- Ransomware. Malicious software that restricts user access to the computer system by encrypting the files or locking down the system while demanding a ransom for its release.
- Rootkit. Malware designed to conceal the existence of other malicious programs.
- Spyware. Computer software that spies and collects sensitive information without permission from a victim's computer. Examples include key-loggers, password grabbers and sniffers.
- Trojan. A Trojan is a type of malicious software that disguises itself as legitimate software to trick users into downloading and installing malware on their systems.
- Virus. Malicious software that can propagate itself from device to device.
- Worm. A type of virus that exploits vulnerabilities of the operating system to spread. The major difference between worms and viruses is the ability of worms to independently self-replicate and spread while viruses depend on human activity.

Malware Analysis: Static Analysis

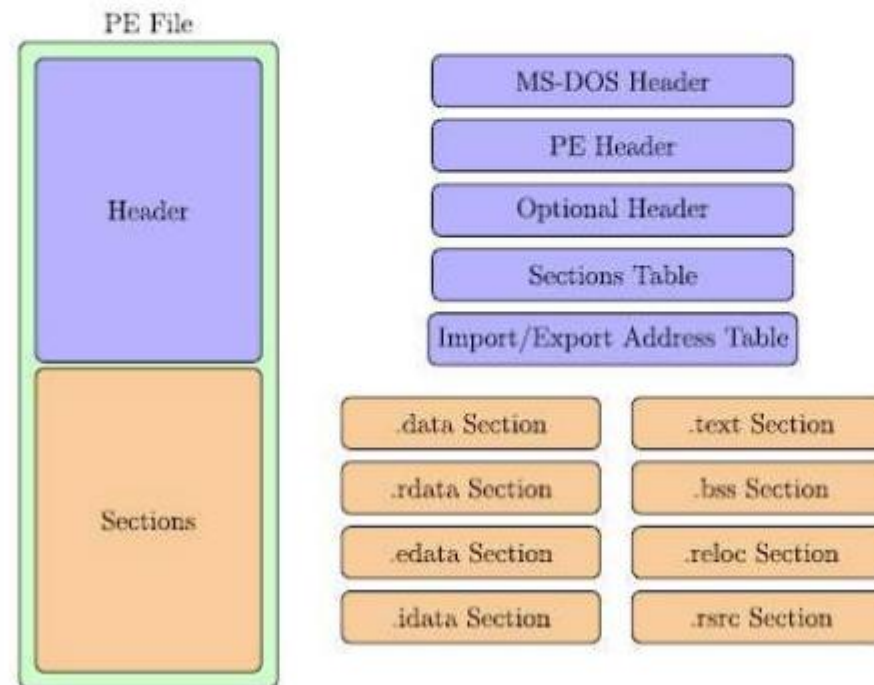
- **Static Analysis** : Examining the code or structure of the executable file without executing it. Produces signature and uses hashing (MD5 and SHA-1)
- **Common static analysis approaches are:**
 - Finding sequence of characters or strings from binary file. (references to file paths of files modified , IP addresses, domain names attack command)
 - Gathering linked libraries and metadata about the file included in the headers
 - Analyze PE file headers and sections . PEView tool
 - Searching for packed and encrypted code
 - Disassembling the program : Translate the machine code into assembly language.

Malware Analysis: Dynamic Analysis

- **Dynamic Analysis** : involves executing the program and monitoring its behaviour on the system.
- **The execution must be carried out in a safe environment:**
 - Physical machines must be set up on air-gapped networks that is isolated from networks to prevent malware from spreading
 - Setup virtual machines to perform dynamic analysis *Vmware station, Oracle Virtualbox*
 - All-in-one software products based on sandbox : *Cuckoo sandbox*
 - Additional utilities : Process Monitor, Process Explorer, Regshot, NetCat and Wireshark
- **Risks:** Some malware can detect when it running in a virtual machine or a sandbox

Portable Executable file format

- PE format is file formats for executables, object codes, DLLs for windows OS.
- **It encapsulates the information necessary for a windows OS to manage the executable code**



Portable Executable file format

- The Portable Executable (PE) format is a file format for executables, object code, DLLs, FON Font files and others used in 32-bit and 64-bit versions of the Windows operating system.
- The PE32 format stands for Portable Executables of 32-bit while PE32 + stands for Portable Executables of 64-bit format.
- Portable Executables encapsulate the information necessary for a Windows operating system to manage the executable_code.
- This includes dynamic library references for linking, API export and import tables, resource management data and thread-local storage data.
- A PE file consists of a number of headers and sections that tell the dynamic linker how to map the file into memory.

Portable Executable file format

- A Portable Executable file has various sections that contain the code and data of the executable including, but not limited to, the following:
- The .data section. This section is used to declare initialized data or constants that do not change at runtime.
- The .bss section. This section is used for declaring variables and contains uninitialized data.
- The .text section. This section keeps the actual code of the program.
- The .rsrc section. This section contains all the resources of the program.
- The .rdata section. This section holds the debug directory which stores the type, size and location of various types of debug information stored in the file.
- The .idata section. This section contains information about functions and data that the program imports from DLLs.
- The .edata section. This section contains the list of the functions and data that the PE file exports for other programs.
- The .reloc section. This section holds a table of base relocations. A base relocation is an adjustment to an instruction or initialized variable value that is needed if the loader could not load the file where the linker assumed it would.

Traditional Machine Learning approach

- Depending on the method of analysis, ML methods can be classified as :
 - Static methods : extract features from static analysis
 - Dynamic methods : extract features from dynamic analysis
 - Hybrid methods : combines both aspect of static and dynamic approach

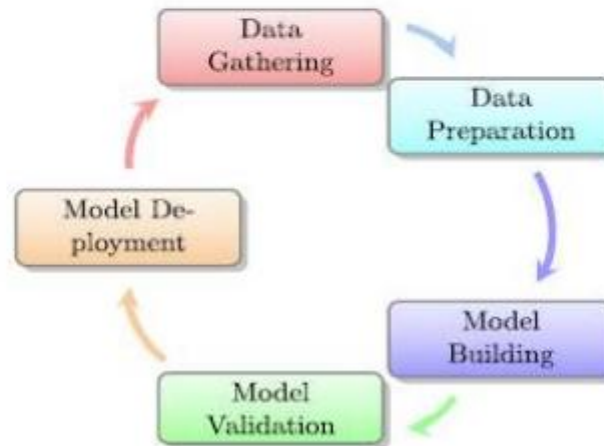


Fig. 2. Machine learning workflow.

Traditional Machine Learning approach

- In machine learning, a workflow is an iterative process that involves gathering available data, cleaning and preparing the data, building models, validating and deploying into production.
- Instead of dealing with raw malware, the data preparation process of traditional machine learning approaches involves preprocessing the executable to extract a set of features that provide an abstract view of the software.
- Afterwards the features are used to train a model to solve the task at hand.
- Because of the variety of malware functionalities, it is important not only to detect malicious software, but also to differentiate between different kinds of malware in order to provide a better understanding of their capabilities.
- The main difference between machine learning solutions for detection or classification of malware is the output returned by the system implemented.
- On the one hand, a malware detection system outputs a single value $y = f(x)$, in the range from 0 to 1, which indicates the maliciousness of the executable.

Traditional Machine Learning approach

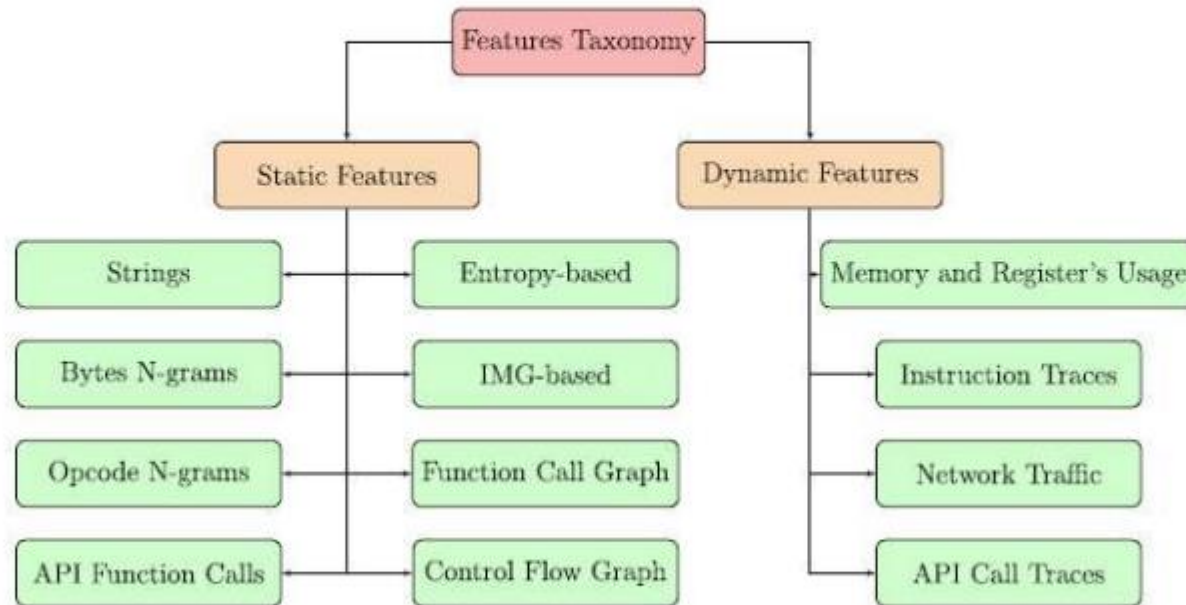


Fig. 3. Taxonomy of features used by traditional M.L. approaches.

Static Features : String analysis

- Static features are extracted from a piece of program without involving its execution. In Windows Portable Executable files, static features are basically derived from two sources of information, the binary content of the executable or the assembly language source file obtained after decompiling and disassembling the binary executable.
- String analysis refers to the extraction of every printable string within an executable or program. A string refers to a sequence of characters.
- Searching for strings is the simplest way to obtain clues about the functionality of a program.
- The information that may be found in these strings can be, for instance, URLs that the program connects to, file locations or filepaths of files accessed/modified by the program, names of the menus of the application, etc.
- The utility called “Strings” can be used to search an executable for ASCII and Unicode strings, ignoring context and formatting. er decompiling and disassembling the binary executable.

Static Features : n-grams

- The most common type of features for malware detection and classification is n-grams.
- An n-gram is a contiguous sequence of n items from a given sequence of text.
- N-grams can be extracted from the bytes sequences representing the malware's binary content and from the assembly language source code.
- By treating a file as a sequence of bytes, byte n-grams are extracted by looking at the unique combination of every n consecutive bytes as an individual feature.
- On the other hand, the sequence of assembly language instructions can also be extracted from the assembly language source code.
- In this case, only the mnemonic of the instruction, i.e. “ADD”, “MUL”, “PUSH”, etc., is retained. Thus, opcode or mnemonic n-grams refer to the unique combination of every n consecutive opcodes as an individual feature.

Static Features : API Call

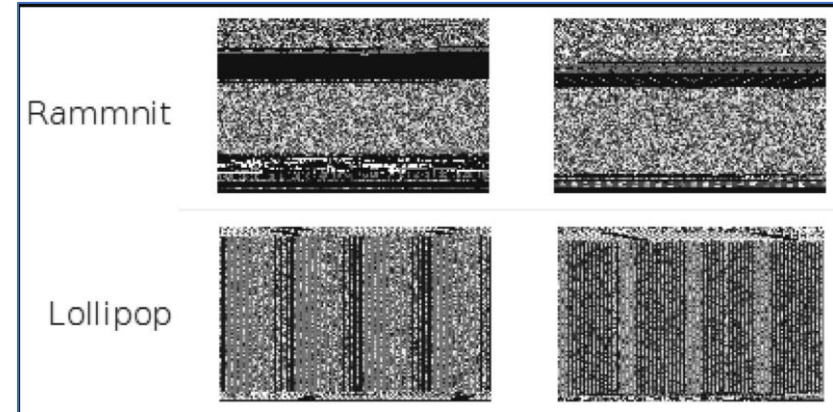
- Application Programming Interfaces (API) and their function calls are regarded as very discriminative features.
- API functions invocation can be used to model the program's behavior.
- Essentially, API functions and system calls are related to services provided by the operating systems such as networking, security, file management, and so on.
- As there is no other way for software to access the system resources without using API functions, the invocation of particular API functions provides key information to represent the behavior of malware.

Static Features : Entropy

- Malware authors often employ a variety of obfuscation techniques to hide the malicious purpose of the executables.
- The two most commonly used are compression and encryption, which are used to conceal malicious segments from static analysis.
- Consequently, it is of great interest for the information security industry to be able to detect the presence of encrypted or compressed segments of code within executable files.
- To this end, entropy analysis has been employed because files with segments of code that have been compressed or encrypted tend to have higher entropy than native code.
- In the context of information theory, the entropy of a bytes sequence reflects its statistical variation.

Static Features : Gray scale image

- It is an interesting approach for malware visualization - visualized the malware's binary content as a gray scale image.
- This is achieved by interpreting every byte as one pixel in an image, where values range from 0 to 255 (0:black, 255:white).
- Afterwards, the resulting array is reorganized as a 2-D array.
- As shown in Fig, samples from two malware families represented as gray scale images.
- One can observe that the image representation of samples of a given family is quite similar while distinct from that belonging to a different family.
- This visual similarity is the result of reusing code to create new binaries.
- Thus, if old samples are re-used to implement new binaries, the resulting ones would be similar.
- In most cases, by representing an executable as a gray scale image it would be possible to detect small variations between samples belonging to the same family.



Static Features : Function call graphs

- A Function Call Graph (FCG) is a directed graph whose vertices represent the functions of which a software program is composed, and the edges symbolize function calls.
- A vertex is represented by either one of the following two types of functions:
 - Local functions, implemented by the programmer to perform specific tasks.
 - External functions: provided by the O.S. or system and external libraries.
- One particularity of the graph is that only local functions can invoke external functions, not the other way around.
- Function call graphs are generated from the static analysis of the disassembly file.
- To extract the FCG of Windows PE executables, IDA Pro or Radare2 can be used.

Static Features : Control Flow Graph

- A Control Flow Graph (CFG) is a directed graph in which the nodes represent basic blocks and the edges represent control flow paths.
- A basic block is a linear sequence of program instructions having an entry point (the first instruction executed) and an exit point (the last instruction executed).
- A CFG is a representation of all the paths that can be traversed during a program's execution.

Dynamic Features : Memory and Register's usage

- Dynamic features are those extracted from the execution of malware at runtime.
- Dynamic analysis involves monitoring malware (and observing the real sequence of instructions executed or the sequence of API functions triggered) as it runs or examining the system after the malware has executed.
- It reveals process creation, file and registry manipulation and modifications of memory values, registers and variables.
- The behavior of a computer program can be represented by the values of the memory contents at runtime.
- In other words, values stored in different registers while a computer program is running can distinguish benign from malicious programs.

Dynamic Features : Instruction traces

- A dynamic instruction trace is a sequence of processor instructions called during the execution of a program.
- Contrary to the static instruction trace, dynamic traces are ordered as they are executed while static traces are ordered as they appear in the binary file.
- Dynamic traces are a more robust measure of the program's behavior, since code packers and encrypters can obfuscate and hinder the code instructions from static analysis.

Dynamic Features : Network traffic

- Detecting malicious traffic on a network can uniquely provide specific insights into the behavior of malicious programs.
- As soon as malware infects a host machine, it may establish communication with an external server to obtain the commands to execute on the victim or to download updates, other malware or to leak private and sensitive information of the user/device.
- As a result, the monitoring of network traffic entering and exiting the network, the traffic within the network and the host activity, provide helpful information to detect malicious behavior.
- Approaches in the literature extract events at several abstraction levels, from raw packets to network flows, detailed protocol decoding such as HTTP and DNS requests, to host-based events and metadata such as IP addresses, ports and packet counts.